
CloudServe Support Automation

Forward Deployed AI Engineering — Capstone Report

Author: Madhav Mehta **System version:** 2.0 **Evaluation run:** the single run recorded in `evaluation/results/`, 80 validation tickets

1. Executive summary

CloudServe Solutions asked for a chatbot. Their support function takes eight to twelve hours to reply against a two-hour service agreement, resolves 42% of tickets at first contact, and has watched satisfaction fall to 3.2 out of 5.

The discovery evidence says a chatbot would solve the wrong problem. **71.4% of incoming tickets are already answered somewhere in the twenty-nine support articles CloudServe maintains, but only 43.8% are resolved at first contact** (Table 1). Nearly half of everything escalated to a senior engineer had a documented answer available. CloudServe are not short of answers. They are short of a way to find them — a conclusion their own technical writer reached before we arrived, and could not get acted on.

What was built is a retrieval and routing system rather than a generative one. It finds the existing answer, cites the article it came from, says plainly when there is no answer, and hands everything else to a person with the analysis already attached. On the recorded run it resolved **75.0%** of tickets without a human, against a 60% target and a 43.8% baseline, with **zero** private data occurrences, **zero** unresolvable citations, and a decision log reconciling exactly 80 records to 80 tickets.

The single most important caveat: the system currently resolves standard-tier customers at 66.67% and business-tier at 86.67% — a twenty-point gap against a governance condition of five. The Project Brief is explicit that governance conditions hold or do not, and that a system failing one is not fit to deploy. **On that measure this system is not yet deployable**, and section 8 sets out why I chose to report the gap rather than engineer it away.

2. The problem

2.1 What was asked for

Marcus Adeyemi, Head of Support, was direct about the request and about its origin: "Honestly? I picture it answering the easy ones so my people can do the hard ones. I do not have a strong view about how it works." He was equally direct about not knowing what his tickets contain: "If you asked me for a breakdown I would be guessing. We have the data, we have never really sat down with it."

That is the gap this project turns on. The request named a mechanism — a chatbot — because the mechanism was the part that could be pictured. It said nothing about where answers come from, whether they are correct, or what happens when there is no answer.

2.2 What the evidence shows instead

Each of the five people interviewed described one edge of the same situation, and none described it whole.

Sofia Restrepo, on tier one: "Seven out of ten I could answer without looking anything up... Finding it and writing it out. We have documentation, and it is good documentation, but searching it is painful, so most of us do not."

Ines Varga, who writes that documentation, knows it goes unused and knows why: "Someone writes *my deployment keeps dying* and my article is called *resolving container health check failures*. There is no path between those two phrases in a keyword search."

Daniel Okonkwo, receiving escalations: "About half of what reaches me is something tier one could have resolved if they had been confident, or if they had found the right page."

The ticket data settles all three. **71.4%** of tickets are labelled answerable from existing documentation — Sofia's seven-in-ten, accurate to within a point and a half. **49.11%** of escalations had a documented answer — Daniel's "about half", accurate. Against that, **43.8%** were resolved at first contact.

2.3 The gap, and why the difference matters

Table 1 — the delivery gap

Measure	Value
Tickets answerable from existing documentation	71.4%
Tickets actually resolved at first contact	43.8%
Delivery gap	27.6 points
Escalations whose answer was already documented	49.11%
Mean satisfaction in the record	2.97 / 5
Repeat contacts within a week	21.6%

A chatbot generates answers. On the 71.4% it would be inventing answers to questions already answered — producing text that may or may not match what Ines wrote, with no way to tell which. On the remaining 28.6% it would be confidently wrong, which is precisely Marcus's stated failure condition: "our customers are engineers, they will screenshot a confidently incorrect answer and put it on the internet within the hour."

The system CloudServe needs retrieves rather than invents, cites so that a wrong answer can be traced to a wrong article or a misread one, and escalates rather than guessing.

3. Discovery findings

All figures below are produced by `evaluation/discovery_analysis.py` over the 500 development tickets, and can be regenerated with one command. The full tables are in the Stage 1 workbook.

Finding 1 — The answers exist; the delivery does not

Covered in section 2. It is the finding the whole design rests on, and it is the reason the system retrieves from a fixed reviewed corpus rather than generating freely.

Finding 2 — Effort and volume are not the same problem

Table 2 — the four most expensive intents

Intent	Share of volume	Share of effort	Mean minutes	Answerable from docs
security_incident	5.2%	9.69%	786	53.8%
compliance_request	5.2%	8.38%	679	65.4%
data_residency	5.8%	7.08%	515	72.4%
feature_request	4.0%	6.87%	724	0%
Total	20.2%	32.0%		

Twenty per cent of volume consumes a third of all effort, and three of those four intents are ones the corpus flags as never-automate. **Automation cannot touch the expensive work.** What it can do is clear the documented, cheap, high-volume tail so that expensive work gets the attention it needs.

This changed the business case I would put to Marcus. It is not "replace agent time"; it is "return agent time to the work that requires a person". It also sets a hard ceiling: no amount of automation improvement touches that 32%.

Finding 3 — A stakeholder was wrong, and the person nobody asked about was worst served

Sofia believed non-fluent customers had the worst outcomes and that nobody had noticed. The record says otherwise: non-fluent tickets resolve at 45.83% against 43.16% for fluent, with higher satisfaction and faster resolution.

Meanwhile **enterprise customers have the worst first-contact resolution of any tier at 37.35%** and much the slowest resolution at 369 minutes. Ravi Menon assumed the opposite — that his enterprise-plan colleague "gets answers in about an hour" — and Marcus worried enterprise "will notice immediately if they get worse service". Both were wrong in the same direction about a gap that already runs against them.

Had I taken Sofia's account on trust I would have built a special path for a problem that does not exist and missed a real one. This is the finding that most changed how I treated the rest of the interviews.

4. Requirements

Ten functional and eight non-functional requirements, each traceable to discovery evidence. The full document is the Stage 2 workbook; the traceability chain matters more than the list.

Table 3 — selected requirements and their origins

Req	Requirement	Evidence	Code	Test
FR-03	Retrieve only from the reviewed corpus; return nothing when nothing fits	Ines on search failure; Daniel on stale snippet files	<code>src/retrieval.py</code>	<code>TestA4Retrieval</code>
FR-04	Route deterministically against a threshold determined from data	Sofia's three escalation criteria; Marcus on confident wrongness	<code>src/router.py</code>	<code>TestA5Routing</code>
FR-05	Grounded, cited answers that disclose they are automated	Ravi: "I calibrate how much I trust it"; Ines on knowing the source	<code>src/generator.py</code>	<code>TestA6Citations</code>
FR-06	Validate every response, with power to block	Daniel on commitments; Governance Framework	<code>src/guardrails.py</code>	<code>TestA7Guardrails</code>
FR-07	Log every decision so it can be reconstructed	Marcus: "I need to be able to say why it did what it did"	<code>src/db.py</code>	<code>TestA8DecisionLog</code>
FR-09	Escalations carry a handover brief	Daniel: "I do not need it to be right. I need it to show its working"	<code>src/router.py</code>	<code>test_escalation_carries_the_brief_and_the_sources</code>
NFR-06	Cross-group variation under 5 points	Added in v2 after the fairness audit found 20	<code>evaluation/metrics.py</code>	—

One requirement was deliberately written against a stakeholder's stated view. Daniel distrusts automating billing queries. The corpus flags none of the 24 as must-not-automate and 87.5% are documented, and his

underlying concern — commitments about money — is enforced directly by a guardrail. Billing queries are automated, and the disagreement is documented rather than silently resolved.

5. Architecture and design

Six components in sequence with three cross-cutting concerns: decision logging on every path, guardrails on input and output, and graceful degradation at every external boundary. `docs/architecture.md` carries the full diagram and every alternative considered.

5.1 The decision that matters most: what the threshold measures

Requirements v1 routed on the intent classifier's confidence at a threshold of 0.80. Two measurements on development data ended that, and they are the substance of this project's revision.

The classifier is accurate and badly under-confident. Out-of-fold intent accuracy is 99.8%, but observed accuracy sits at or near 100% in every confidence band from 0.1 upward while stated confidence ranges from 0.14 to 0.82. Expected calibration error: **39.79 points**, against a requirement of five. With 22 near-uniform classes, logistic regression spreads probability thinly and reports 0.08 for a prediction that is correct.

And the number barely related to the decision. Sweeping it from 0.05 to 0.80 moved automation precision by under two points. Whether a ticket is answerable from documentation turns out to be largely independent of how sure the classifier is about its topic — unsurprising once you notice that `answerable_from_docs` ranges from 0% to 96% *across* intents.

What replaced it: a separate binary model trained on `expected_route`, wrapped in isotonic calibration, estimating directly the probability that a ticket can be resolved without a person. Expected calibration error **4.24 points**, inside the requirement. Intent still drives the never-automate policy rules; it no longer drives the threshold.

5.2 Choosing the threshold

T = **0.45**, selected by minimising modelled cost rather than by tuning until a headline figure looked acceptable.

The costs come from the interviews. Marcus: an escalation "costs us roughly four times what a resolved one costs". A correct automation is 1, an escalation is 4. The third cost — a wrong automated answer — he characterises but does not quantify, so it is a parameter, set at 12, published with sensitivity.

Table 4 — sensitivity to the one figure we had to assume

Assumed cost of a wrong automation	Chosen T	Automated	Escalated	Precision
4	0.05	82.4%	17.6%	75.24%
8	0.45	75.8%	24.2%	77.57%
12	0.45	75.8%	24.2%	77.57%
20	0.90	3.6%	96.4%	88.89%
40	0.95	1.4%	98.6%	85.71%

The choice is stable between 8 and 12 and then collapses. **The threshold is not robust to an assumption CloudServe has never been asked to make**, and getting a figure from them is the most valuable half-hour available before deployment.

5.3 Retrieval, and an argument that measurement did not support

Ines's account is a clear argument for semantic retrieval over keyword search, and the Project Brief recommends Chroma with sentence embeddings. Retrieval was therefore built as a hybrid with three tiers — Chroma embeddings, a local latent-semantic index, and TF-IDF — fused by reciprocal rank fusion, using the best tier available and always reporting which.

All three score identically on this corpus: 89.08% hit rate at 3, 85.15% at 1, MRR 0.8711. Twenty-nine articles is too small for semantic matching to show a benefit; and the latent-semantic tier is a projection of the TF-IDF matrix, so it cannot add information TF-IDF did not have. Only the transformer tier would genuinely test Ines's hypothesis, and it could not be benchmarked in the development environment.

The honest conclusion is narrower than the architecture implies: the hybrid structure costs nothing and is in place, but on this corpus TF-IDF does the work, and the case for semantic retrieval rests on an interview rather than a measurement. That is outstanding work, not a result.

The Chroma tier is deliberately excluded from `requirements.txt`. A1 is a pass-or-fail gate, and a clean checkout must not fail because a large optional download would not install on the assessor's machine.

6. Implementation

Python 3.10+, 61 tests grouped by acceptance criterion, no required services and no required network access.

What was difficult. Retrieval took twice its estimate, because chunking strategy was reworked twice before section-based chunking with a whole-article chunk settled it. Routing took over twice its estimate entirely because of the calibration discovery.

A bug worth recording. An early version of the hybrid fusion normalised fused scores to 0–1, which handed the best candidate a perfect score *even when the query matched nothing at all* — a query of pure gibberish returned a confident top result, breaking A4's requirement that retrieval return nothing rather than something irrelevant. It was caught by `demo.py`, which states what each scenario expects and compares it against the live result. The relevance gate now runs on raw similarity before fusion.

That bug is the argument for the demonstration script checking itself rather than printing a success line. The previous version of this project printed "All 3 Scenarios Executed Cleanly!" unconditionally.

What I would restructure. The pipeline's `_finish` method carries too many parameters; the output object should be assembled incrementally. And the readiness model and the intent classifier both build their own TF-IDF vectoriser over the same text, which is wasteful.

7. Evaluation

7.1 Method

One run, over 80 validation tickets, on the date recorded in `evaluation/results/metrics_report.json` under `run_context.run_started_at`. **The hidden evaluation set was run once.** Development used the 500 development tickets only; all calibration and threshold selection used out-of-fold scoring on that set, so no ticket was scored by a model that trained on it.

Every figure is computed by the system. The markdown report's pass and fail verdicts come from a single function comparing each measured value against its target — no verdict is written by hand anywhere, and `test_status_flags_are_computed_not_hardcoded` feeds the reporter a deliberately terrible result and fails if anything returns PASS.

7.2 Results

Table 5 — business outcomes, which lead because they are what CloudServe buys

Measure	Baseline	Target	Achieved	Confidence	Status
First contact resolution	42%	$\geq 60\%$	75.0%	Medium — see 7.4	PASS
Escalation rate	58%	$\leq 30\%$	25.0%	Medium	PASS
Mean reply, whole queue	8–12 hrs	< 5 min	54.5 min	Low — projection	FAIL
Mean reply, automated only	—	—	5 ms	High — measured	—
Satisfaction	3.2 / 5	4.0	not reported	—	—

Table 6 — technical performance

Measure	Target	Achieved	Status
Classification macro precision	$\geq 85\%$	100%	PASS
Classification macro recall	—	100%	—
Retrieval hit rate (n=53)	—	90.57%	—
Citation accuracy (n=60)	$\geq 95\%$	96.67%	PASS
Citations that did not resolve	—	0	—
Hallucination rate (proxy)	$\leq 5\%$	0%	PASS
Latency p95	< 3 s	5.42 ms	PASS

Table 7 — governance conditions

Condition	Requirement	Observed	Status
Private data in responses	0	0	PASS
Decision log reconciliation	exact	80 / 80, 0 duplicates	PASS
Cross-group variation	< 5 pts	20.0 pts	FAIL
Confidence calibration	within 5 pts	4.24 pts	PASS

Condition	Requirement	Observed	Status
Guardrail probes	all behave	10 / 10	PASS

Eight of ten targets met, two missed.

7.3 What the numbers mean for CloudServe

Resolution moves from 43.8% to 75.0%, which on 500 tickets a week is roughly **156 additional tickets resolved without a person, each week**. At Marcus's own arithmetic — an escalation costs four times a resolved ticket — that is worth about 468 units of cost a week against the current baseline.

The more useful effect may be the one that is harder to count. Half of what reaches Daniel currently need not have; every escalation now arrives with a likely intent, the alternatives considered, the relevant articles with match scores, and a plain statement of what the system was unsure about. He estimated he would be "twice as fast" with exactly that.

What has not improved. Mean reply time across the whole queue is 54.5 minutes against a five-minute target. Automated replies land in milliseconds; the blended figure assumes escalated tickets still wait the historic median of 218 minutes for an agent. **Automation alone cannot move this number below five minutes.** Even at 100% automation of everything automatable, the 25% that escalates would hold the mean above 50 minutes. Reaching the target requires agent response time to fall as well, which is a staffing and process question rather than a software one. Reporting only the automated figure would have made this look solved when it is not.

7.4 How far these figures should be trusted

The figures above should be treated with caution because:

1. **The corpus is synthetic and unusually separable.** 100% classification precision is a property of this data, not a claim about CloudServe's traffic. Out-of-fold accuracy on 500 development tickets was 99.8%, which no real 22-class support corpus would produce. I would expect real traffic in the 70–85% range, and the routing threshold would need re-deriving at that point.
2. **Citation accuracy and hallucination rate are automated proxies.** They confirm a citation resolves to a passage retrieval returned; they do not confirm the passage supports the sentence it is attached to. The Evaluation Framework asks for human review of at least fifty responses by two independent assessors. That was not done at that scale — I reviewed a sample of twenty myself, which is one assessor and under half the sample size, and found no unsupported claim. That is weaker evidence than the 0% suggests.
3. **Reply time is a projection, not a measurement.** No human replied to any ticket in this run.
4. **Satisfaction is not reported at all.** There were no customers. The previous version of this system reported a CSAT of 4.4 produced by the formula `4.4 if fcr >= 50 else 3.8`, which is a number invented and presented as a measurement. Its absence is more honest than its presence.
5. **Automation precision is measured against `expected_route`**, which encodes what an annotator judged correct. A ticket automated against an `escalate` label has not necessarily had a wrong answer — only one the corpus would have preferred a human to give.
6. **Small segments.** The enterprise segment in the fairness audit has eight tickets and should carry very little weight.

8. Governance and risk

The full framework is `04_Submission/MadhavMehta_Governance_Framework.md`.

Decision logging. One record per ticket on every path, carrying the Governance Framework's minimum schema plus channel, customer tier and language fluency. Those three additions make the fairness audit a query against the log rather than a separate exercise — which is the difference between an audit that runs on every evaluation and one that ran once, for this report.

Guardrails. Six checks on every response, all able to block, none disableable by a flag. Private data is blocked and escalated rather than redacted and sent: redaction hides the fault while leaving it in place. Because the validation corpus contains no attacks, ten engineered probes run through the same pipeline instance after every run — 10 of 10 behaved as required, including a control confirming an acceptable response is not blocked.

The failing condition. Standard-tier customers are resolved at 66.67% against 86.67% for business tier: twenty points against a requirement of five. The router does not read customer tier except for one enterprise-urgency rule, so this is emergent, arising from what standard-tier customers write about rather than from how they are treated.

I chose not to fix it, and the reason matters. The fix within reach — tier-aware thresholds — would close the measured gap by applying a different standard according to what a customer pays. Ravi Menon's warning was precisely that a widening gap between plans "will come up at renewal". Improving the metric while making the underlying treatment more explicitly unequal would be worse than reporting the number. The right fix is to understand why standard-tier tickets are harder, and I did not have time for it.

The most likely remaining harm is a confident, well-cited answer drawn from an article that has become wrong since it was last reviewed. Every control here checks that an answer is *grounded*; none checks that the ground is still true. `last_reviewed_days_ago` exists in the corpus and the system does not use it.

9. The requirements revision

Seven revisions; the full log is the Stage 5 workbook.

The substantive one is section 5.1: the routing signal was replaced after calibration measurement showed the number v1 thresholded on was off by 39.79 points and nearly irrelevant to the decision anyway. It cost about seven hours and most of week two's contingency.

What triggered it was running the full validation set unattended on day four of week two rather than day five, which the Build Specification suggests. That single day of margin is the difference between finding the problem and shipping it.

What it taught me is that I had treated a number as meaningful because it was in the range 0 to 1 and had "confidence" in its name. I built a threshold on it, wrote a requirement around it, and justified a value of 0.80 — none of which involved checking whether it measured anything. The Evaluation Framework mentions the calibration table almost in passing; producing it invalidated a component. I would now plot the calibration curve the day a score enters a decision path, not the week I write the evaluation.

Six further assumptions were falsified, including a stakeholder's vivid account of a fairness problem that the data contradicted, and my own expectation that semantic retrieval would beat keyword search.

10. Declaration of AI tool use

Anthropic Claude was used throughout as a pair-programming and review assistant. This declaration is specific because a vague one is not worth making.

Used substantially for: drafting and refactoring implementation code across `src/` and `evaluation/`, including the calibration and metrics modules; generating the regex pattern families in `src/guardrails.py`; structuring the test suite; drafting prose in this report and the workbooks; and review passes that identified inconsistencies between documents and code.

Not used for: the problem statement in section 2 and the Stage 1 workbook; the interpretation of the evaluation results in section 7.3 and 7.4; the decision not to fix the fairness gap and the reasoning in section 8; and the reflection in section 9 and the Stage 5 workbook. These are the judgement calls and they are mine.

Where its output was overridden. It initially proposed retaining intent-confidence thresholding with a recalibration layer; I replaced the signal entirely after the sweep showed it was close to irrelevant to the decision, not merely miscalibrated. It proposed reporting a satisfaction proxy; I removed satisfaction reporting altogether rather than publish a number derived from a formula. It proposed tier-aware thresholds to bring the fairness variation inside the requirement; I declined, for the reason in section 8.

A note on provenance. An earlier iteration of this project contained AI-drafted material I had not checked against the code: a report describing a routing algorithm that did not exist, file paths that were never created, and a metrics table with "PASS" written into every row regardless of the measured value. That is the failure mode of using these tools without verifying their output, and correcting it is a substantial part of what version 2.0 is.

11. Conclusions

The main conclusion is about framing rather than technology. CloudServe asked for a system that produces answers, and what they needed was a system that finds them. The distinction is not academic: it determined the corpus restriction, the citation requirement, the decision to escalate rather than guess, and the choice of a retrieval architecture over a generative one. The evidence for it — 71.4% answerable against 43.8% resolved — took an afternoon to establish and is the single most valuable thing in this project.

The second conclusion is that measurement changes designs, but only if it happens early enough to act on. Running the full set on day four of week two rather than day five is why this report contains a genuine revision.

What I would do next, in order

1. **Close the twenty-point resolution gap between customer tiers** — or get an explicit, written decision from CloudServe that it is acceptable. This is the one thing that would stop me deploying.

2. **Ask Marcus what a wrong automated answer costs.** The threshold is not robust above about 20×, and this is a half-hour conversation.
3. **Use `last_reviewed_days_ago`.** The most likely remaining harm is a perfectly grounded answer drawn from a stale article, and the corpus already carries the field that would flag it.
4. **Instrument repeat contacts.** 21.6% historically, nobody watches it, and it is the only measure that would reveal tickets being closed without being solved.
5. **Benchmark the transformer retrieval tier** where the model host is reachable, and settle whether Ines's argument holds.
6. **Re-derive everything on real traffic.** The classification figures will not survive contact with it, and the threshold will need re-deriving when they change.

What remains uncertain

Whether 71.4% holds on live traffic. Whether `expected_route` reflects CloudServe's actual policy or one annotator's judgement — the system is optimised against it either way. Whether the fairness gap is a property of the tickets or of the system. And whether a support team that has worked around broken search for years will trust a system that finds things for them; Sofia's concern was never accuracy but being made to apologise for an answer she did not write, and no measurement in this report addresses that.